

INSTITUTE OF ENGINEERING AND MANAGEMENT, KOLKATA

ASSIGNMENT 8

Data-Driven Automation (DDT)

Name :	Ashmita Saha
Enrollment no. :	12023002028118
Department :	CSE (AI & ML)
Course :	Python for Automation
Academic Session :	2023-2027
Submission Date :	11 Sept, 2026

1. MODULE NAME & EXPERIMENT TITLE

Module Name: Web Automation with Selenium & Data-Driven Testing Frameworks

Experiment Title: Implementation of Data-Driven Automation (DDT) using Selenium WebDriver and Pandas in Python

2. PROBLEM STATEMENT

Build an automated login script that reads multiple test cases from an external source (such as a CSV or Excel file using pandas). The execution flow must iterate through all test data records to perform login operations on the target application with varying combinations of valid and invalid usernames and passwords, asserting that proper validation errors or success page redirects occur for each test case.

3. OBJECTIVE

- To separate test execution logic from test data, promoting reusability and maintainability through Data-Driven Testing (DDT).
- To dynamically parse external data parameters from a CSV file (`login_data.csv`) using the Python pandas library.
- To automate web page elements interaction (input fields, click events) on `https://www.saucedemo.com/` using Selenium WebDriver.
- To implement dynamic assertions verifying both successful authentications and appropriate error message responses.

4. TOOLS, SOFTWARE & CONCEPTS USED

Python 3.x

Selenium WebDriver

Pandas Library

Google Chrome / ChromeDriver

Data-Driven Testing (DDT)

CSS Selectors & Element ID Locators

Assertions & Validation

- **Data-Driven Testing (DDT):** A software testing methodology where test input data and expected results are stored in table/spreadsheet formats, allowing a single test script to execute across all records.
- **Pandas DataFrames:** Used for seamless reading and iteration over structural dataset records (`pd.read_csv` and `iterrows()`).
- **Selenium Webdriver & Locators:** Utilized `By.ID` and `By.CSS_SELECTOR` to locate interactive elements such as `user-name`, `password`, `login-button`, and error message banners.

5. SOURCE CODE & IMPLEMENTATION STEPS

Step 1: Test Dataset Configuration (`login_data.csv` / `login_data.py` representation)

The external CSV dataset contains various valid and invalid credential combinations along with expected assertion results:

login_data.csv

login_data.csv screenshot

Step 2: Main Automation Script (assignment8.py)

The main script reads the CSV via Pandas, initializes the Chrome driver, and loops over each row to execute authentication checks:

assignment8.py

assignment8.py screenshot

6. OUTPUT & EXECUTION RESULTS

Upon running `python assignment8.py`, the script iterates through all 4 test cases provided in the CSV file and outputs execution logs into the terminal:

```
$ python assignment8.py
Test Passed: Login successful
Test Passed: Invalid login rejected
Test Passed: Invalid login rejected
Test Passed: Invalid login rejected

Process finished with exit code 0
```

Test Case #	Username	Password	Expected Result	Execution Status
TC_01	standard_user	secret_sauce	success	PASSED (URL contains / inventory.html)
TC_02	standard_user	wrong_password	error	PASSED (Error message validated)
TC_03	wrong_user	secret_sauce	error	PASSED (Error message validated)
TC_04	wrong_user	wrong_password	error	PASSED (Error message validated)

7. RESULT, OBSERVATION & CONCLUSION

Result: All four test scenarios defined in the external dataset were executed sequentially. The framework correctly identified successful logins and verified proper error handling for invalid credential sets.

Observation: Utilizing Pandas to drive Selenium script execution eliminates redundant script creation for multiple login permutations. Reading from external sources makes test maintenance straightforward without altering the core automation logic.

Conclusion: Data-Driven Automation significantly enhances test coverage and maintenance efficiency. The experiment successfully demonstrated reading external CSV test data and performing automated assertion checks using Python, Selenium, and Pandas.